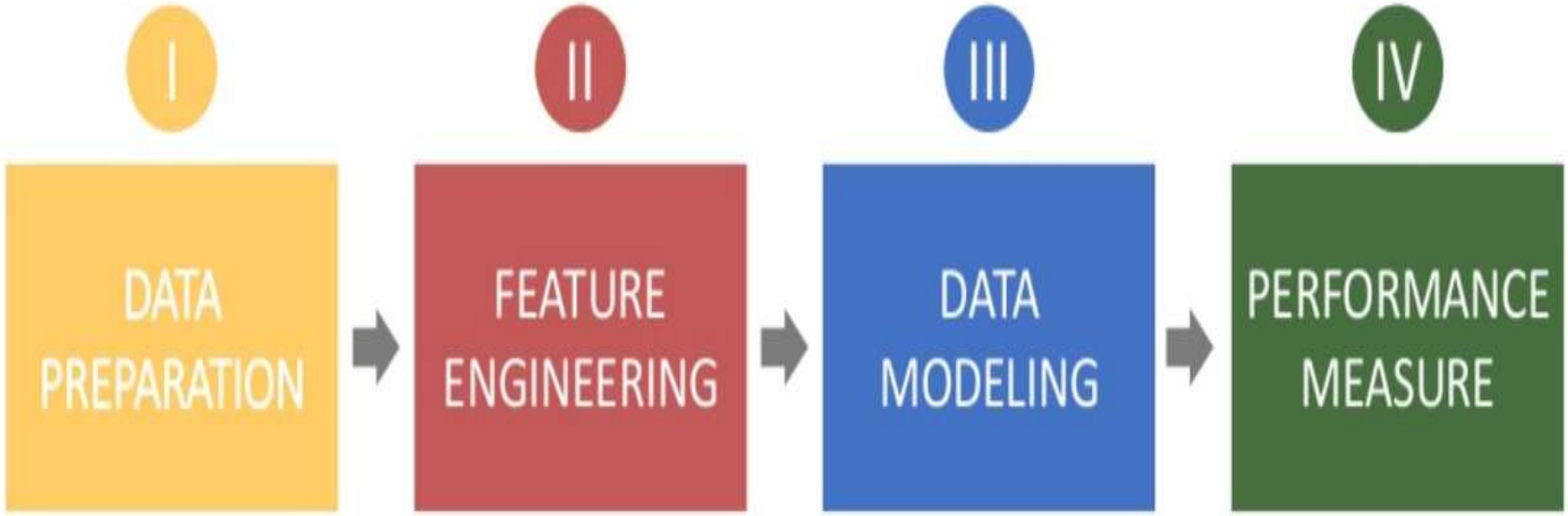
Unit – 2.1

- Introduction to Feature Engineering,
- Feature transformation,
- Feature subset selection,
- Importance of Bayesian methods,
- Bayes' theorem,
- Concept learning through Bayes' theorem,
- Bayesian Belief Network

There are **4 steps** to build a machine learning model...



What is Feature?

- A **feature is an attribute of a data set that is used in a machine learning process.**
- A **feature is an individual measurable property of a phenomenon (fact) being observed.**
- **A feature is a numeric representation of raw data.**
- There are many ways to turn raw data into numeric measurements, which is why features can end up looking like a lot of things.
- Naturally, **features must derive from the type of data that is available.**
- This **features in a data set are also called its dimensions**.
- So **data set having 'n' features is called an n-dimensional data set.**

- The number of features is also important.

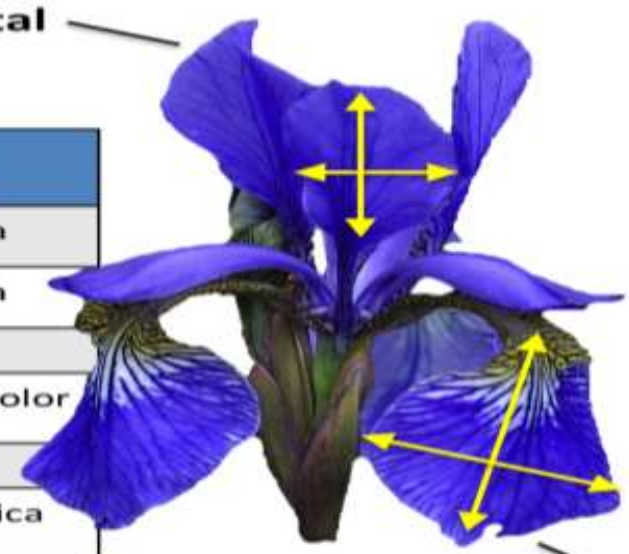
- If there are **not enough informative features**, **then the model will be unable to perform the ultimate task.**

- If there are **too many features**, or if most of them are irrelevant, **then the model will be more expensive and tricky to train**.

Example of feature

Samples
(instances, observations)

	Sepal length	Sepal width	Petal length	Petal width	Class label
1	5.1	3.5	1.4	0.2	Setosa
2	4.9	3.0	1.4	0.2	Setosa
...					
50	6.4	3.5	4.5	1.2	Versicolor
...					
150	5.9	3.0	5.0	1.8	Virginica



Class labels
(targets)

Features
(attributes, measurements, dimensions)

- It has **five attributes or features namely** Sepal.Length, Sepal.Width, Petal.Length, Petal.Width and Species (**Class Label**).
- Species represents the class variable and the remaining fetures are the predictor variables.
- **It is a five dimensional data set.**

What is Feature Engineering?

- Feature engineering refers to **the process of translating a data set into features** such that **these features are able to represent the data set more effectively** and **result in a better learning performance.**

- Another we can say that feature engineering **is transforming data to create model inputs.**

- Feature engineering is **the process of transforming raw data into relevant features**, i.e. that are:
  - **Informative** (it provides useful data for your model to correctly predict the label)
  - **Discriminative** (it will help your model make differences among your training examples)
  - **Non-redundant** (it does not say the same thing than another feature),

resulting in improved model performance on unseen data.

Example: Predict the price of an apartment in Paris

	YES ✓	NO ✗
Informative?	Size in square meters	The name of your neighbor (unless it's Brad Pitt)
Discriminative?	Size in square meters	Simple or double glazed window? (99% is double glazing in Paris)
Non-redundant?	Size in square meters	Size in square inches

Is the feature ...

Obviously a good feature to predict the price of an apartment !

After feature engineering, your dataset will be a big matrix of numerical values.

n features

Value of feature #n for training example #1

$$X = \begin{array}{c} \begin{array}{cccc} & \text{Feature \#1} & \text{Feature \#2} & \text{Feature \#n} \end{array} \\ \begin{pmatrix} 1 & x_{1,1} & x_{1,2} & & x_{1,n} \\ 1 & x_{2,1} & x_{2,2} & & x_{2,n} \\ 1 & x_{3,1} & x_{3,2} & \cdots & x_{3,n} \\ \dots & \dots & \dots & & \dots \\ 1 & x_{m-1,1} & x_{m-1,2} & & x_{m-1,n} \\ 1 & x_{m,1} & x_{m,2} & & x_{m,n} \end{pmatrix} \end{array}$$

m training examples

LABELS

$$Y = \begin{pmatrix} y_1 \\ y_2 \\ y_3 \\ \dots \\ Y_{m-1} \\ y_m \end{pmatrix}$$

Remember that **behind "data"** there are two very different notions, training examples and features.

Elements of Feature Engineering

- Feature Engineering is an important pre-processing step for machine learning. It has two major elements.

- **1. Feature Transformation**
- **2. Feature subset selection.**

Feature Transformation

- It transforms the data – **structured or unstructured, into a new set of features which can represent the underlying(basic) problem which machine learning is trying to solve.**

- Feature transformation **is also know as feature engineering**, which means **creating new features from existing features** which may **help in improving the model performance**.

- There are **two variants of feature transformation**:
  - **Feature Construction**
  - **Feature Extraction**

- **<u>Feature Construction :</u>**
- It is the process **discovers missing information about the relationships between features** and **expand the feature space by creating additional features.**

- Hence, **if there are 'n' features** or **dimensions in a data set**, after feature construction '**m' more features or dimensions may get added.**

- So at the end, **the data set will become 'n + m' dimensional**.

- **Feature Extraction :**
- It is the process of extracting or creating a new set of features from the original set of features using some functional mapping.

- Feature extraction is the name for methods that select and /or combine variables into features, effectively reducing the amount of data that must be processed.

- **PCA (principal component analysis)** algorithm is used for feature extraction.

Feature subset selection

- Feature subset selection (or simply feature selection) no new feature is generated.
- Feature selection **is the process of selecting the most relevant features among your existing features.**
- To keep "relevant" features only, we will remove features that are: -
  - **Non informative**
  - **Non discriminative**
  - **Redundant (unnecessary)**

1. Feature Transformation

Feature Transformation

- Feature transformation is used as an effective tool for **dimensionality reduction** and hence for boosting learning model performance.

- Feature transformation is the **process of transforming a feature into a new one with a specific function.**

- There are **two distinct goals of feature transformation:**
  - **Achieving best reconstruction** of the original features in the data set.
  - **Achieving highest efficiency** in the learning task.

1. Feature Construction

- Feature reconstruction involves **transforming a given set of input features to generate a new data set of more powerful features.**

- **Feature construction** methods may be applied to pursue **two distinct goals:**
  - **Reducing data dimensionality.**
  - **Improving prediction performance.**

Example of Feature Construction

apartment_ length	apartment_ breadth	apartment_ price
80	59	23,60,000
54	45	12,15,000
78	56	21,84,000
63	63	19,84,000
83	74	30,71,000
92	86	39,56,000

→

apartment_ length	apartment_ breadth	apartment_ area	apartment_ price
80	59	4,720	23,60,000
54	45	2,430	12,15,000
78	56	4,368	21,84,000
63	63	3,969	19,84,500
83	74	6,142	30,71,000
92	86	7,912	39,56,000

Example of Feature Construction

- The data set has three features - **apartment length**, **apartment breadth**, and **price of the apartment**.
- If it is used as an input to a regression problem, such data can be training data for the regression model.
- So given the training data, **the model should be able to predict the price of an apartment whose price is not known** or **which has just come up for sale.**
- However**, instead of using length and breadth of the apartment as a predictor**, it is much convenient and makes more sense to **use the area of the apartment. which is not an existing feature of the data set.**
- So such a feature, **namely apartment area, can be added to the data set**.
- In other words, we **transform the three-dimensional data set to a four-dimensional data set**, with the **newly 'discovered' feature apartment area being added to the original data set.**

- **Note:** Though for the goal of simplicity the features apartment length and apartment breadth have been retained in Figure.
- in reality, **it makes more sense to reject these features when building the model.**

- There are certain situations **where feature construction is an essential activity before we can start with the machine learning task.**
- These situations are
  - when **features have categorical value and machine learning needs numeric value inputs.**
  - when **features having numeric (continuous) values and need to be converted to ordinal values.**
  - **when text-specific feature construction needs to be done.**

Encoding categorical (nominal) variables

- Let's take the example of another data set on athletes, as presented in Figure 4.3a.
- Say the **data set has features age**, **city of origin**, **parents athlete** (i.e. indicate whether any one of the parents was an athlete) and **Chance of Win**.
- The feature chance of a win is a class variable while the others are predictor variables.
- We know that any machine learning algorithm, **whether it's a classification algorithm (like ANN) or a regression algorithm, requires numerical figures to learn from.**
- So there are three features – **City of origin, Parents athlete, and Chance of win, which are categorical in nature and cannot be used by any machine learning task.**

Age (Years)	City of origin	Parents athlete	Chance of win
18	City A	Yes	Y
20	City B	No	Y
23	City B	Yes	Y
19	City A	No	N
18	City C	Yes	N
22	City B	Yes	Y

(a)

Age (Years)	origin_ city_A	origin_ city_B	origin_ city_C	parents_ athlete_Y	parents_ athlete_N	win_ chance_Y	win_ chance_N
18	1	0	0	1	0	1	0
20	0	1	0	0	1	1	0
23	0	1	0	1	0	1	0
19	1	0	0	0	1	0	1
18	0	0	1	1	0	0	1
22	0	1	0	1	0	1	0

(b)

Age (Years)	origin_city_A	origin_city_B	origin_city_C	parents_athlete_Y	win_chance_Y
18	1	0	0	1	1
20	0	1	0	0	1
23	0	1	0	1	1
19	1	0	0	0	0
18	0	0	1	1	0
22	0	1	0	1	1

(c)

IG. 4.3
eature construction (encoding nominal variables)

- In this case, feature construction can be used to create new dummy features which are usable by machine learning algorithms.
- Since the feature 'City of origin' has three unique values namely City A, City B, and City C, three dummy features namely origin_city_A, origin_city_B, and origin_city_C is created.
- In the same way, dummy features parents_athlete_Y and parents_athlete_N are created for feature 'Parents athlete' and win_chance_Y and win_chance_N are created for feature 'Chance of win'.
- The dummy features have value 0 or 1based on the categorical value for the original feature in that row.
- For example, the second row had a categorical value 'City B' for the feature 'City of origin'.
- So, the newly created features in place of 'City of origin', i.e. origin_city_A,.origin_city_B and origin_city_C will have values 0, 1 and 0, respectively.
- In the same way. parents_athlete_Y and parents_athlete_N will have values 0 and 1.
- respectively in row 2 as the original facture 'Parents athlete' had a categorical value 'No' in row 2.The entire set of transformation for athletes' data set is shown in Figure 4.3b.

- However, examining closely, we see that the features '**Parents athlete**' and '**Chance of win**' in the original data set **can have two values only.**

- So creating **two features from them is a kind of duplication**, since **the value of one feature can be decided from the value of the other.**

- **To avoid this duplication,**we can just leave one feature and eliminate the other, as shown in Figure 4.3c.

Encoding categorical (ordinal) variables

- Let's take an example of a student data set. Let's assume that there are three variable — **science marks, maths marks** and **grade** as shown in Figure 4.4a.

- As we can see, the grade is an ordinal variable with values A, B,C, and D.

- To transform this variable to a numeric variable, we can create a feature num_grade mapping a numeric value against each ordinal value.

- In the context of the current example. grades A, B,C, and D in Figure 4.4a is mapped to values 1,2, 3, and 4 in the transformed variable shown in Figure 4.4b.

marks_science	marks_maths	Grade
78	75	B
56	62	C
87	90	A
91	95	A
45	42	D
62	57	B

(a)

marks_science	marks_maths	num_grade
78	75	2
56	62	3
87	90	1
91	95	1
45	42	4
62	57	2

(b)

FIG. 4.4

Feature construction (encoding ordinal variables)

Transforming numeric (continuous) feature to categorical values

- Sometimes there **is a need of transforming a continuous numerical variable into a categorical variable.**
- For example, we may want to treat the real estate price prediction problem, which is a regression problem, as a real estate price category prediction, which is a classification problem.
- In that case, we can '**bin' the numerical data into multiple categories based on the data range.**
- In the context of the real estate price prediction example, the original data set has a numerical feature apartment_price as shown in Figure 4.5a.
- It can be **transformed to a categorical variable price-grade either as shown in Figure 4.5b or as shown in Figure 4.5c.**

apartment_ area	apartment_ price
4,720	23,60,000
2,430	12,15,000
4,368	21,84,000
3,969	19,84,500
6,142	30,71,000
7,912	39,56,000

(a)

apartment_ area	apartment_ grade
4,720	Medium
2,430	Low
4,368	Medium
3,969	Low
6,142	High
7,912	High

(b)

apartment_ area	apartment_ grade
4,720	2
2,430	1
4,368	2
3,969	1
6,142	3
7,912	3

(c)

FIG. 4.5
Feature construction (numeric to categorical)

Text-specific feature construction

- In the current world, text is maybe the most major medium of communication.
- Whether we think about social networks like Facebook or micro-blogging channels like Twitter or emails or short messaging services such as Whatsapp, text plays a major role in the flow of information.
- Hence, **text mining is an important area of research**— not only for technology practitioners but also for industry practitioners.
- However, making sense of text data, due to the inbuilt unstructured nature of the data, is not so simple.
- In the first place, the text data chunks that we can think about do not have readily available features, like structured data sets, on which machine learning tasks can be executed.
- **All machine learning models need numerical data as input.**
- So **the text data in the data sets need to be transformed into numerical features.**

- Text data, or amount which is the more popular keyword, **is converted to a numerical representation following a process is known as vectorization.**

- In this process, **word occurrences in all documents belonging to the amount are merged in the form of bag-of-words.**

- There are three major steps that are followed:
  - **1. tokenize**
  - **2. count**
  - **3. normalize**

- In order to tokenize a amount, **the blank spaces and punctuations are used as delimiters to separate out the words, or tokens.**
- Then the **number of occurrences of each token is counted, for each document.**
- Lastly, **tokens are weighted with reducing importance when they occur in the majority of the documents.**
- A **matrix is then formed with each token representing a column and a specific document of the corpus representing each row.**
- **Each cell contains the count of occurrence of the token in a specific document.**
- This **matrix is known as a document-term matrix (also known as a term-document matrix).**
- Figure 4.6 represents a typical document-term matrix which forms an input to a machine learning model.

This	House	Build	Feeling	Well	Theatre	Movie	Good	Lonely	...
2	1	1	0	0	1	1	1	0	
0	0	0	1	1	0	0	0	0	
1	0	0	2	1	1	0	0	1	
0	0	0	0	1	0	1	1	0	
.	.	.	.	.	.	.	.	.	
.	.	.	.	.	.	.	.	.	
.	.	.	.	.	.	.	.	.	

FIG. 4.6

Feature construction (text-specific)

2. Feature Extraction

- In feature extraction, **new features are created from a combination of original features**, Some of the commonly used operators for combining the original features include

- **1. For Boolean features:** Conjunctions, Disjunctions, Negation, etc.
- **2. For nominal features:** Cartesian product. M of N. etc.
- **3. For numerical features:** Min, Max, Addition, Subtraction, Multiplication,Division, Average, Equivalence, Inequality. etc.

Let's take an example and try to understand. Say, we have a data set with a feature set F_i ($F_1, F_2, \ldots, F_n$). After feature extraction using a mapping function f ($F_1, F_2, \ldots, F_n$) say, we will have a set of features $F_i'(F_1', F_2', \ldots, F_m')$ such that $F_i' = f(F_i)$ and $m < n$. For example, $F_1' = k_1F_1 + k_2F_2$. This is depicted in Figure 4.7.

$Feat_A$	$Feat_B$	$Feat_C$	$Feat_D$
34	34.5	23	233
44	45.56	11	3.44
78	22.59	21	4.5
22	65.22	11	322.3
22	33.8	355	45.2
11	122.32	63	23.2

→

$Feat_1$	$Feat_2$
41.25	185.80
54.20	53.12
43.73	35.79
65.30	264.10
37.02	238.42
113.39	167.74

$$Feat_1 = 0.3 \times Feat_A + 0.9 \times Feat_A$$
$$Feat_2 = Feat_A + 0.5\ Feat_B + 0.6 \times Feat_C$$

FIG. 4.7
Feature extraction

Terminology

- Variance
- Covariance & Correlation
- Eigenvectors & Eigenvalues
- Principal Components

Variance

- **Variance** measures how far a set of data is spread out.
- A **variance** of zero indicates that all of the data values are identical (equal).
- A high **variance** indicates that the data points are very spread out from the mean, and from one another.
- **Variance** is the average of the squared distances from each point to the mean.

$$s^2 = \frac{\sum_{i=1}^{n} (X_i - \overline{X})^2}{(n-1)}$$

Covariance & correlation

- When comparing data samples from different populations, **covariance** is used to determine how much two random **variables** vary together, A positive **covariance** means that asset returns move together while a negative **covariance** means they move inversely.

- whereas **correlation** is used to determine when a change in one variable can result in a change in another.

- Both **covariance and correlation measure** linear relationships between **variables**.

Terminology (Covariance)

- How *two* dimensions vary from the mean with respect to each other

$$\mathrm{cov}(X,Y) = \frac{\sum_{i=1}^{n}(X_i - \bar{X})(Y_i - \bar{Y})}{(n-1)}$$

  - cov(X,Y) > 0: Dimensions increase together
  - cov(X,Y) < 0: One increases, one decreases
  - cov(X,Y) = 0: Dimensions are independent

Covariance Matrix

- Contains covariance values between all possible dimensions:

$$C^{nxn} = (c_{ij} \mid c_{ij} = \mathrm{cov}(Dim_i, Dim_j))$$

- Example for three dimensions (x,y,z) (Always symetric):

$$C = \begin{pmatrix} \mathrm{cov}(x,x) & \mathrm{cov}(x,y) & \mathrm{cov}(x,z) \\ \mathrm{cov}(y,x) & \mathrm{cov}(y,y) & \mathrm{cov}(y,z) \\ \mathrm{cov}(z,x) & \mathrm{cov}(z,y) & \mathrm{cov}(z,z) \end{pmatrix}$$

cov(x,x) → variance of component x

Eigenvector & Eigenvalue

- Eigenvector is a **direction of the line passing through the origin.**
- Eigenvalue is a **number telling how much variance there is data in that direction.**
- **Every eigenvector has a corresponding eigenvalue.**
- Eigenvector which the **highest eigenvalue is therefore the principal component.**
- **Dimensional data = eigenvectors/values.**

Feature Extraction Algorithms

1. PCA (Principal Component Analysis)

- Objectives of unsupervised learning methods:
  - **Reduce dimensionality**
  - **Score all observations**
  - **Cluster similar observations together**
- Well-known linear transformation methods:
  - **PCA**, Factor Analysis, Projection Pursuit,etc.

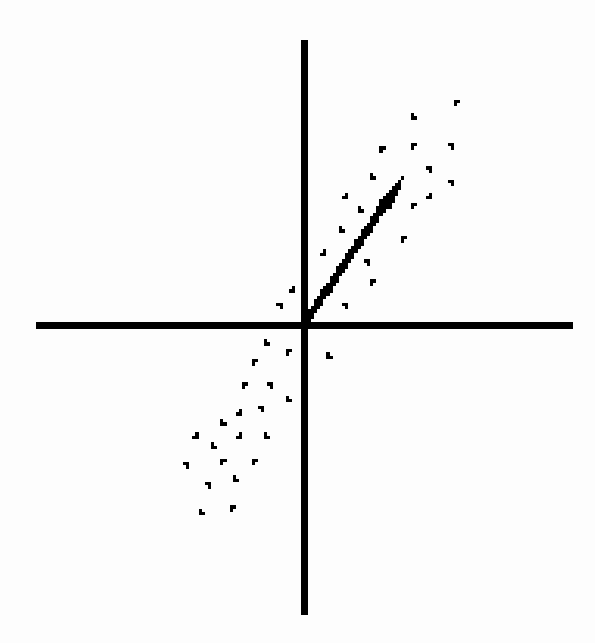
What is PCA?

- **Transform some large number of variables into a smaller number of uncorrelated variables** called principal components (PCs).
- **Developed to capture as much of the variation in data as possible.**
- The extracted uncorrelated components are called *principal components(PC)*
- **Eestimated from the eigenvectors of the covariance** or **correlation matrix of the original variables**.
- The projections of the data on the eigenvectors.
- Eextracted by linear transformations of the original variables so that the first few PC's contain most of the variations in the original dataset.

Objectives of PCA (Principal Component Analysis)

- Principal Component Analysis is a technique used to:
  - **Reduce the dimensionality of the data set**
  - **Identify new meaningful underlying variables**
  - **Loose minimum information**

- by finding the directions in which a cloud of data points is stretched most

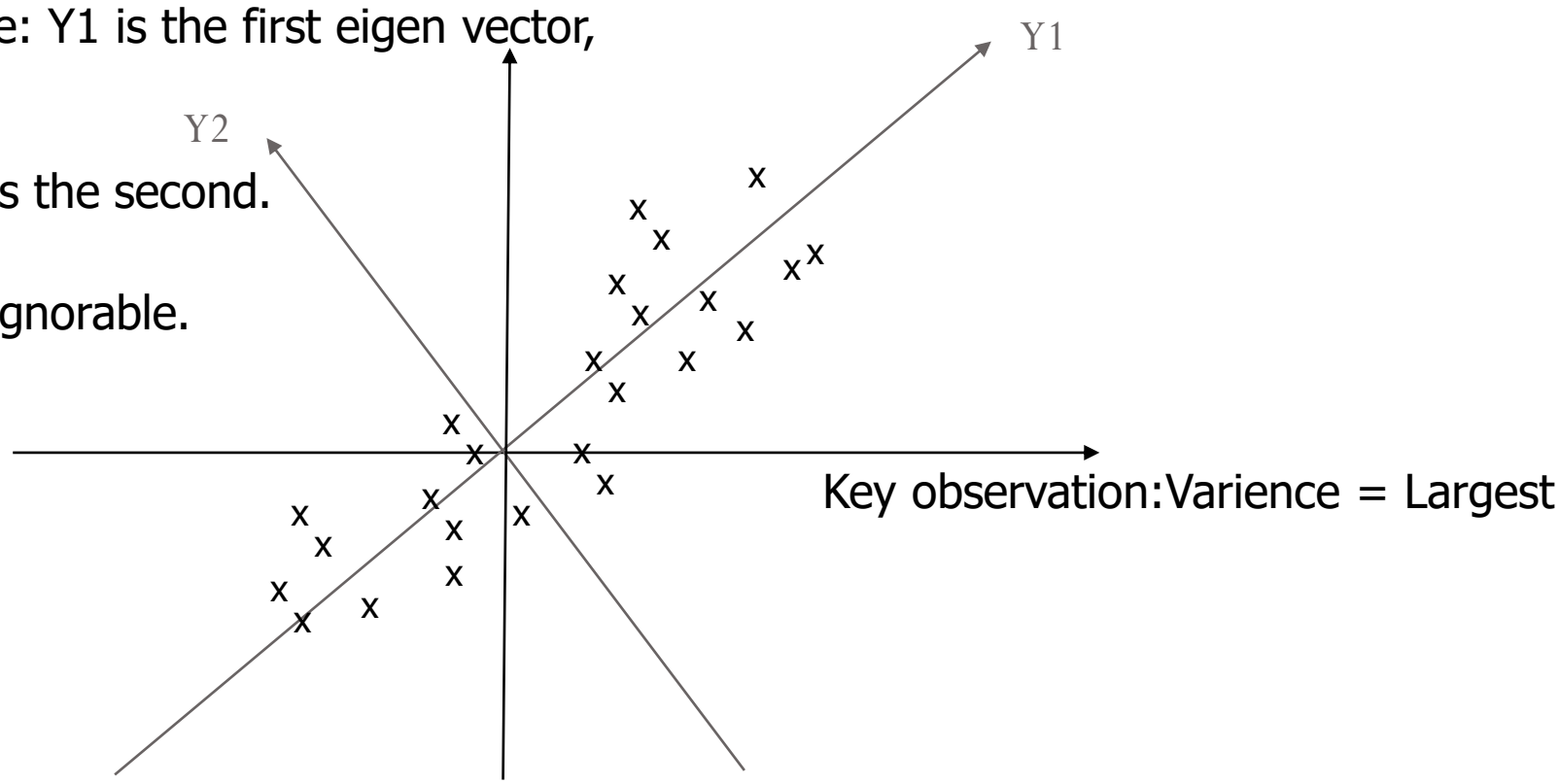


Principal Component Analysis

Note: Y1 is the first eigen vector,

Y2 is the second.

Y2 ignorable.



Principal Component Analysis: one attribute first

Question: how much spread is in the data along the axis? (distance to the mean)

Variance=Standard deviation^2

$$s^2 = \frac{\sum_{i=1}^{n} (X_i - \overline{X})^2}{(n-1)}$$

Temperature
42
40
24
30
15
18
15
30
15
30
35
30
40
30

Now consider two dimensions

Covariance: measures the correlation between X and Y

- **cov(X,Y)=0:** independent
- **Cov(X,Y)>0:** move same direction
- **Cov(X,Y)<0:** move opposite direction

$$\mathrm{cov}(X,Y) = \frac{\sum_{i=1}^{n}(X_i - \bar{X})(Y_i - \bar{Y})}{(n-1)}$$

X=Temperature	Y=Humidity
40	90
40	90
40	90
30	90
15	70
15	70
15	70
30	90
15	70
30	70
30	70
30	90
40	70
30	90

More than two attributes: covariance matrix

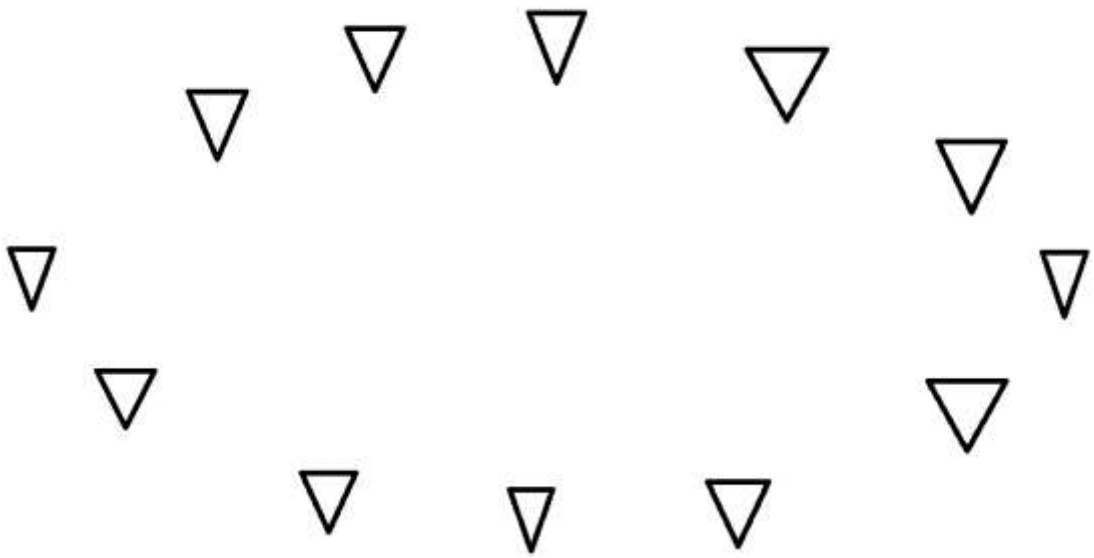
- Contains covariance values between all possible dimensions (=attributes):

$$C^{nxn} = (c_{ij} \mid c_{ij} = \mathrm{cov}(Dim_i, Dim_j))$$

- Example for three attributes (x,y,z):

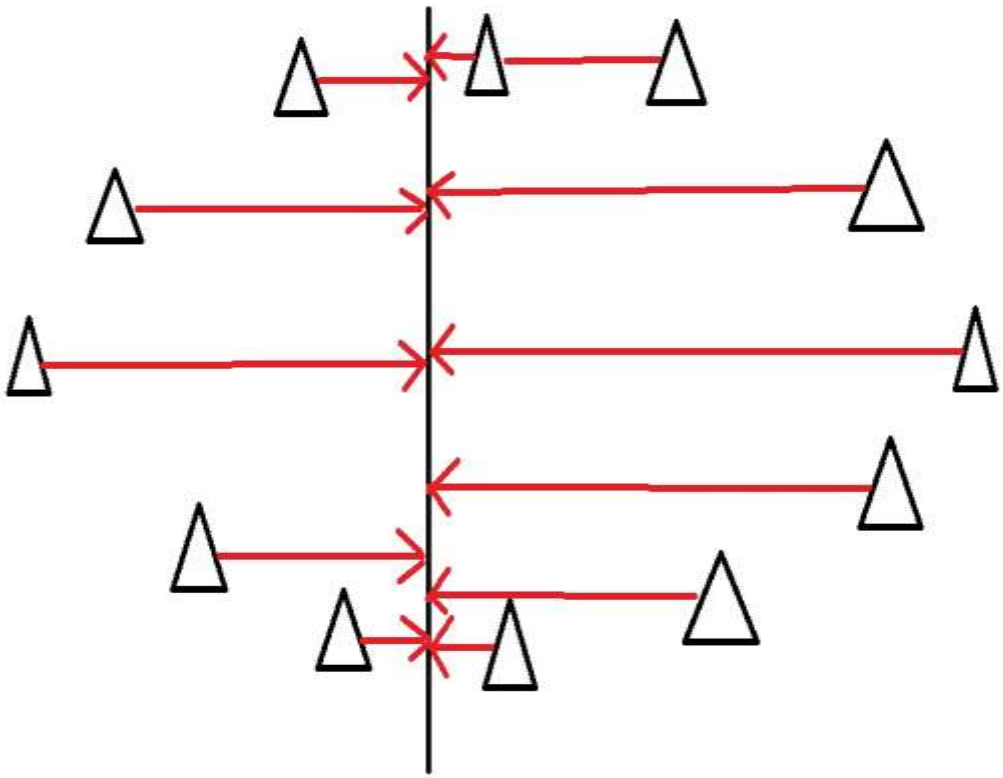
$$C = \begin{pmatrix} \mathrm{cov}(x,x) & \mathrm{cov}(x,y) & \mathrm{cov}(x,z) \\ \mathrm{cov}(y,x) & \mathrm{cov}(y,y) & \mathrm{cov}(y,z) \\ \mathrm{cov}(z,x) & \mathrm{cov}(z,y) & \mathrm{cov}(z,z) \end{pmatrix}$$

What is Principal Component Analysis?

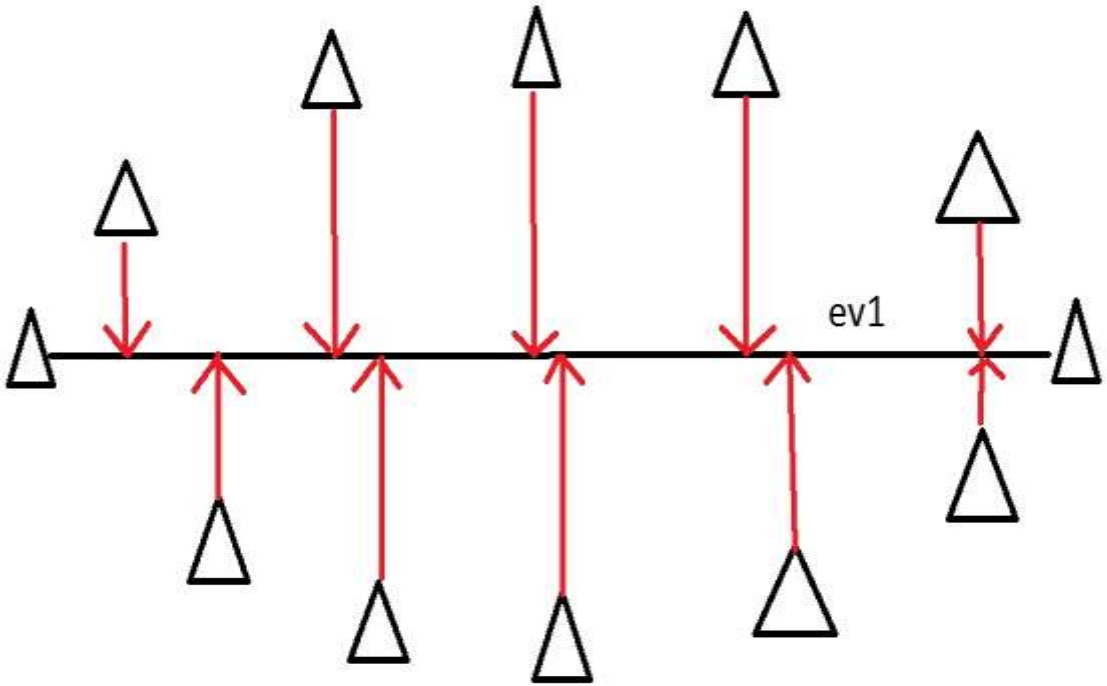


- They are the directions where there is the most variance, the directions where the data is most spread out.

To find the direction where there is most variance, find the straight line where the data is most spread out when projected onto it. A vertical straight line with the points projected on to it will look like this:



•On this line the data is way more spread out, it has a large variance.
In fact there isn't a straight line you can draw that has a larger variance than a horizontal one. A horizontal line is therefore the principal component in this example.



Eigenvalues & Eigenvectors

- Vectors $\mathbf{x}$ having same direction as $A\mathbf{x}$ are called *eigenvectors* of A (A is an n by n matrix).
- In the equation $A\mathbf{x}=\lambda\mathbf{x}$, λ is called an *eigenvalue* of A.

$$\begin{pmatrix} 2 & 3 \\ 2 & 1 \end{pmatrix} x \begin{pmatrix} 3 \\ 2 \end{pmatrix} = \begin{pmatrix} 12 \\ 8 \end{pmatrix} = 4x \begin{pmatrix} 3 \\ 2 \end{pmatrix}$$

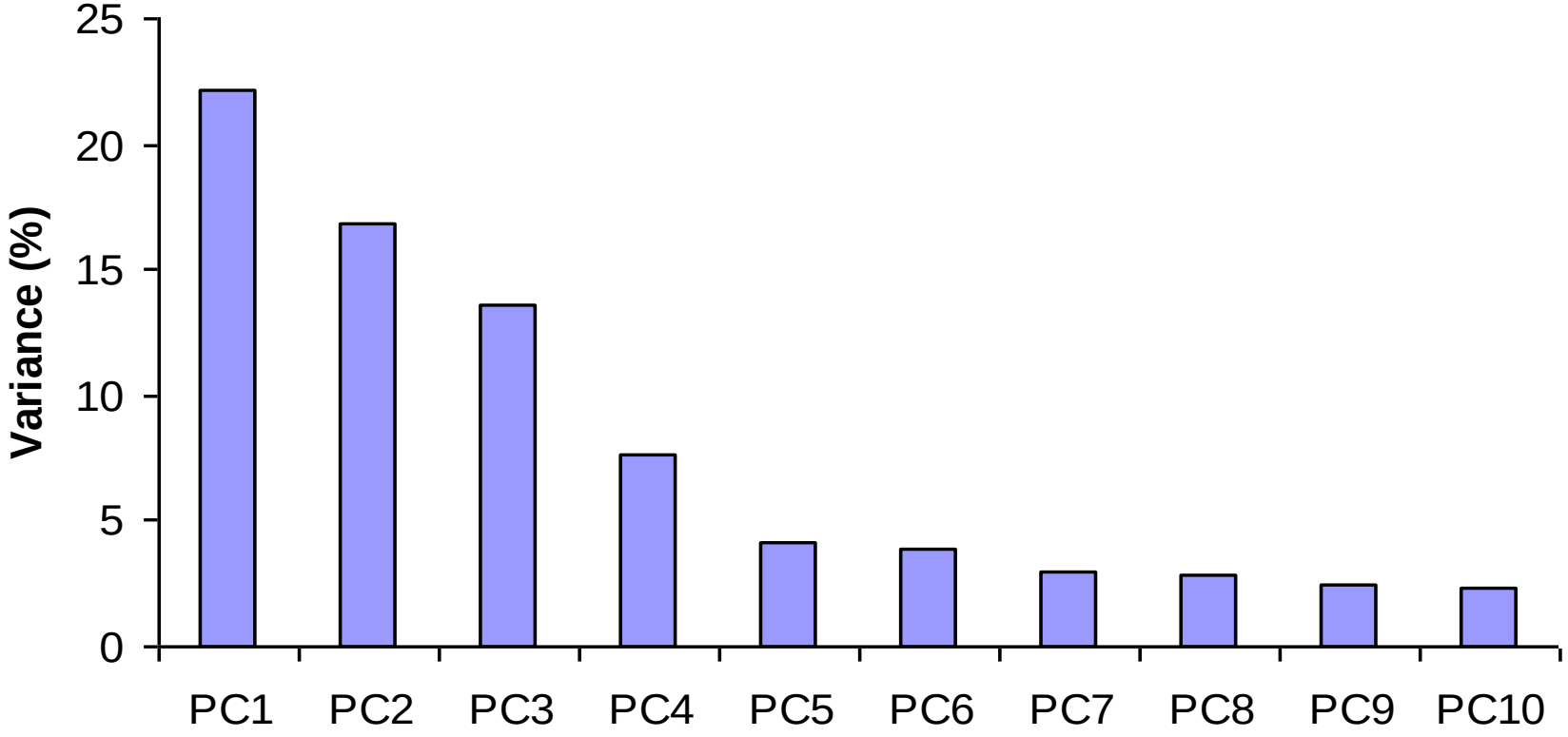
Principal components

- **1. principal component (PC1)**
  - The eigenvalue with the largest absolute value will indicate that the data have the largest variance along its eigenvector, the direction along which there is greatest variation

- **2. principal component (PC2)**
  - the direction with maximum variation left in data, orthogonal to the 1. PC

- In general, only few directions manage to capture most of the variability in the data.

Principal components - Variance



Steps to do PCA

- PCA works based on a process called eigenvalue decomposition of a covariance matrix of a data set. Below are the steps to be followed:
- 1. First, **calculate the covariance matrix of a data set.**
- 2. Then, **calculate the eigenvalues of the covariance matrix.**
- 3. The eigenvector **having highest eigenvalue represents the direction in which there is the highest variance**. So this will help in identifying the first principal component.
- 4. The cigenvector having the next **highest eigenvalue represents the direction in which data has the highest remaining variance and also orthogonal to the first direction.** So this helps in identifying the second principal component.
- 5. Like this, identify the **top 'k' eigenvectors having top 'k' eigenvalues so as to get the 'k'principal components.**

Application of feature reduction

- **Face Recognition**
- **Image Compression**
- **Gene Expression Analysis**
- **Data Reduction**
- **Data Classification**
- **Trend Analysis**
- **Factor Analysis**
- **Noise Reduction**

Feature reduction algorithms

- **Unsupervised**
  - Latent Semantic Indexing (LSI): **truncated SVD**
  - Independent Component Analysis (ICA)
  - Principal Component Analysis (PCA)
  - Canonical Correlation Analysis (CCA)

- **Supervised**
  - **Linear Discriminate Analysis (LDA)**

- **Semi-supervised**
  - Research topic

Singular value decomposition

- Singular Value Decomposition (SVD) is a common dimensionality reduction technique in data science.
- SVD is the **decomposition** of a matrix A **into 3 matrices – U, S, and V**
- Singular value decomposition (SVD) **is a matrix factorization technique commonly used in linear algebra**.
- **SVD of a matrix A (m * n) is a factorization of the form:**

$$A = U \pounds V$$

- where, U and V are orthonormal matrices, **U** is an m * m unitary matrix**,V** is an n * n unitary matrix and **£** is an m x n rectangular diagonal matrix.
- The diagonal entries of **£** are known as singular values of matrix A.
- The columns of U and V are called the left-singular and right-singular vectors of matrix A. respectively.
- **SVD is generally used in PCA, once the mean of each variable has been removed.**
- Since it is not always advisable to remove the mean of a data attribute, especially when the data is sparse (as in case of text data),
- **SVD is a good choice for dimensionality reduction in those situations.**

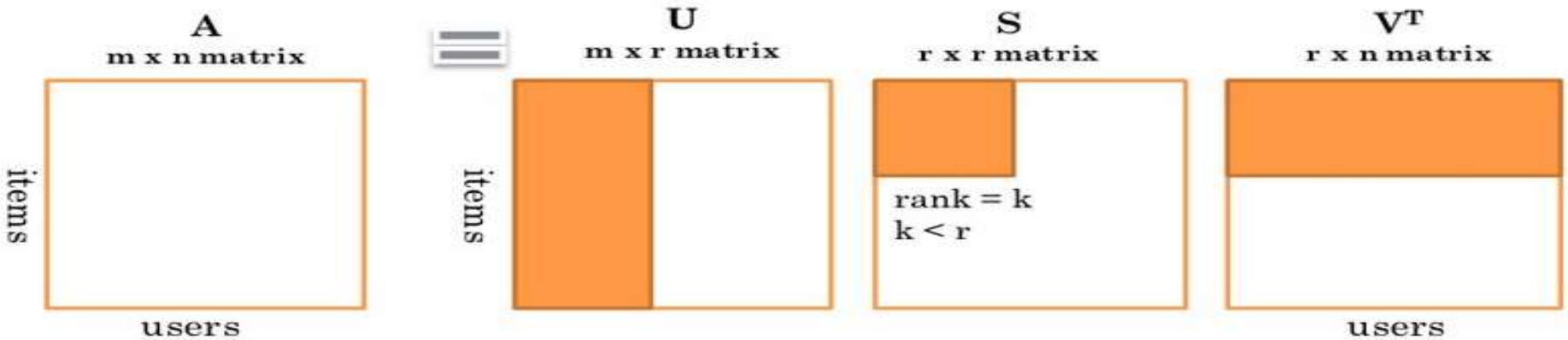
- SVD of a data matrix is expected to have the properties highlighted below:
  - 1. Patterns in the attributes are captured by the right-singular vectors, i.e. the columns of V.
  - 2. Patterns among the instances are captured by the left-singular, i.e. the columns of U.
  - 3. Larger a singular value, larger is the part of (he matrix A that it accounts for and its associated vectors,
  - 4, New data matrix with 'k' attributes is obtained using the equation

$$\mathbf{D' = D * [v1, v2 \ldots, vk]}$$

- Thus, the dimensionality gets reduced to k
- **SVD is often used in the context of text data.**

Singular Value Decomposition (SVD)

$$A = U \times S \times V^T$$



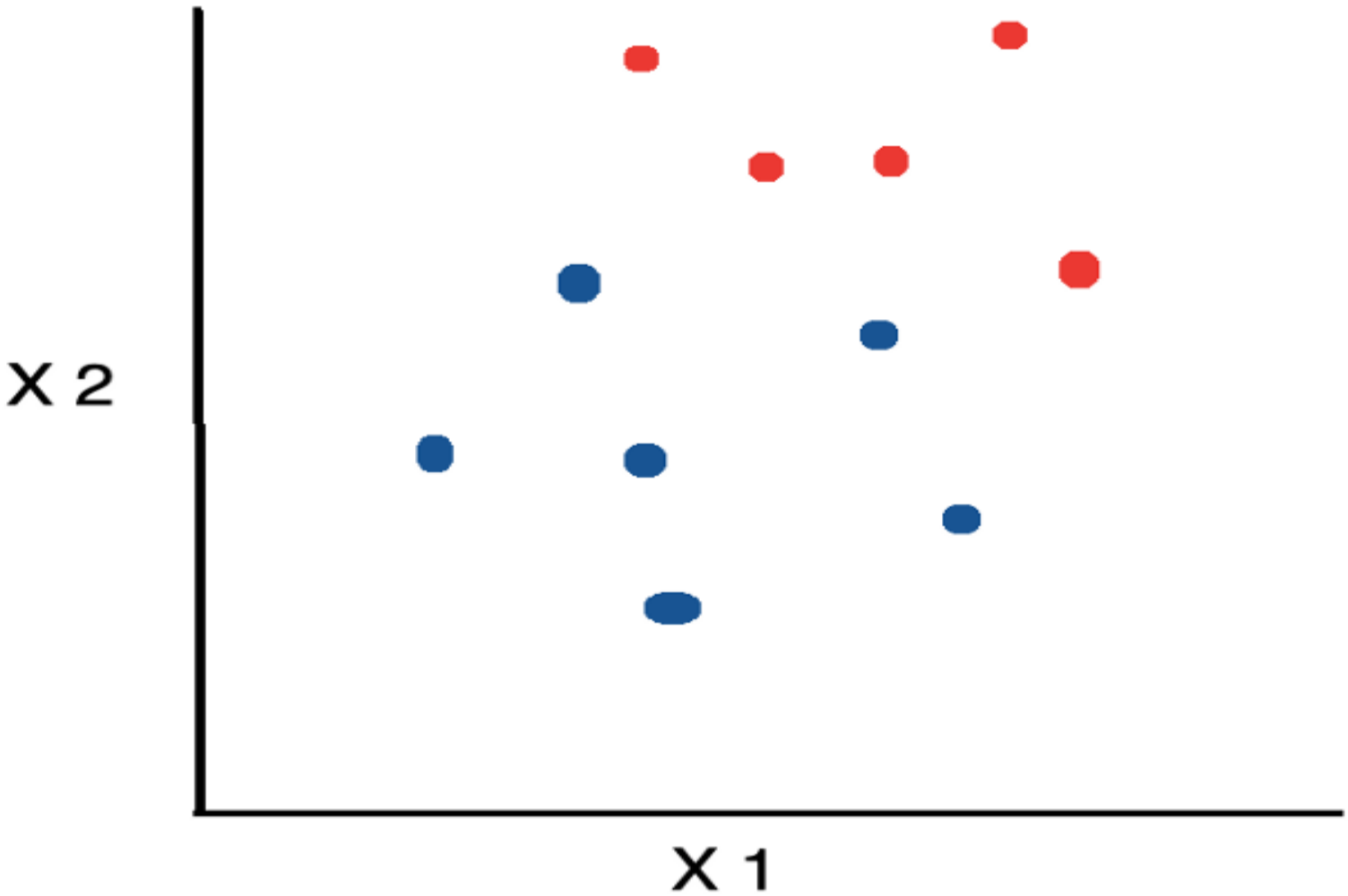
$$A_k = U_k \times S_k \times V_k^T$$

- S is the diagonal matrix of singular values. Think of **singular values** as the importance values of different features in the matrix
- The **rank** of a matrix is a measure of the unique information stored in a matrix. Higher the rank, more the information
- **Eigenvectors** of a matrix are directions of maximum spread or variance of data

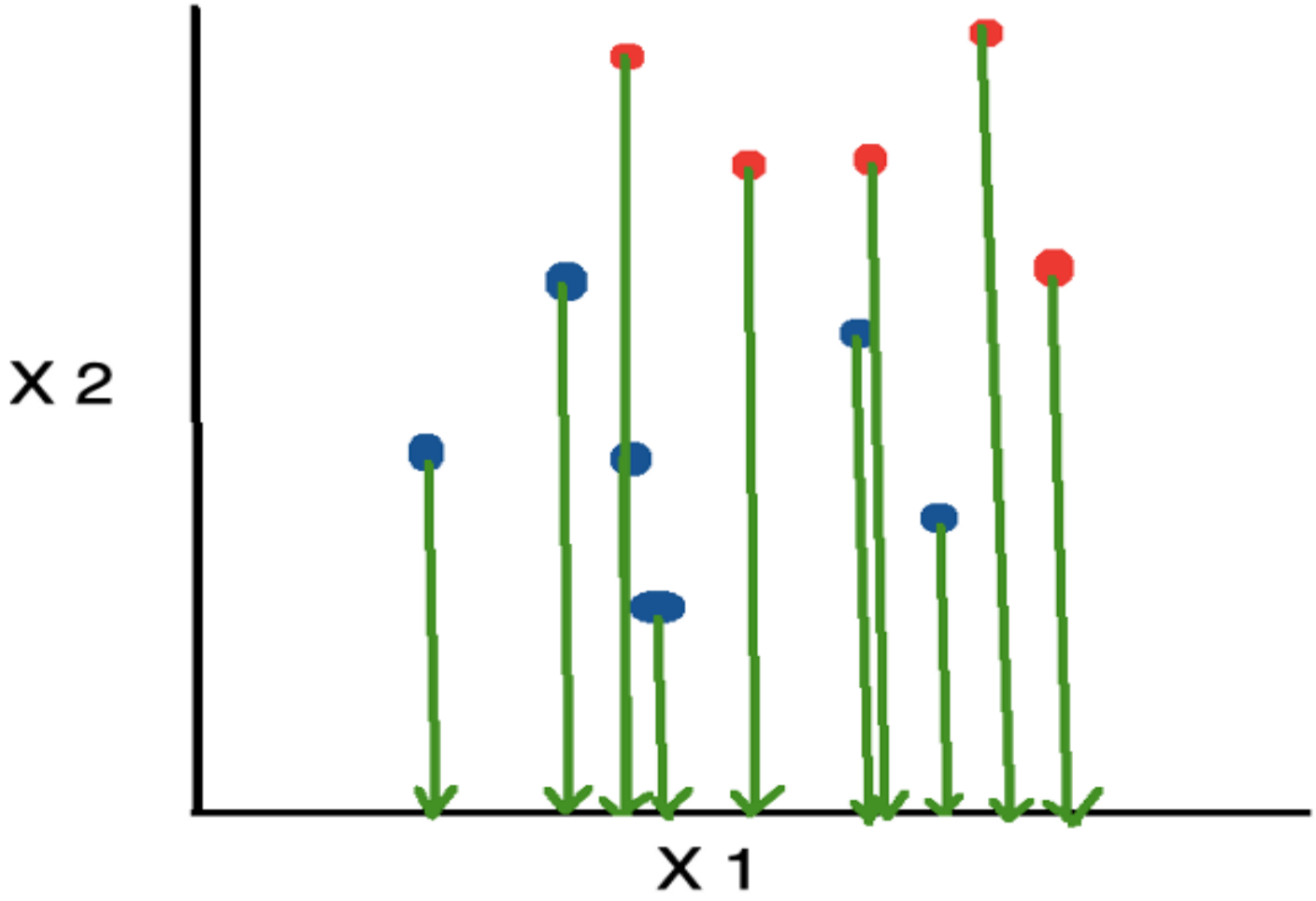
Linear Discriminant Analysis

- Linear Discriminant Analysis (LDA) is a **dimensionality reduction technique.**
- Linear discriminant analysis (LDA) is **another commonly used feature extraction technique like PCA or SVD.**
- As the name implies dimensionality reduction **techniques reduce the number of dimensions (i.e. variables) in a dataset while retaining as much information as possible.**
- The objective of LDA is similar to the sense that it intends to transform a data set into a lower dimensional feature space.
- However, **unlike PCA, the focus of LDA is not to capture the data set variability.**
- Instead, LDA focuses on class separability, i.e. separating the features based on class separability so as to avoid over-fitting of the machine learning model.

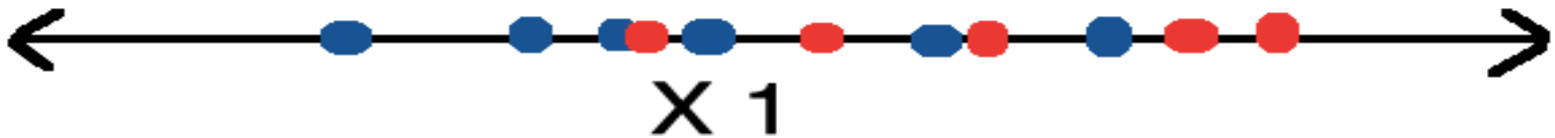
suppose that we plotted the relationship between two variables where each color represent a different class.



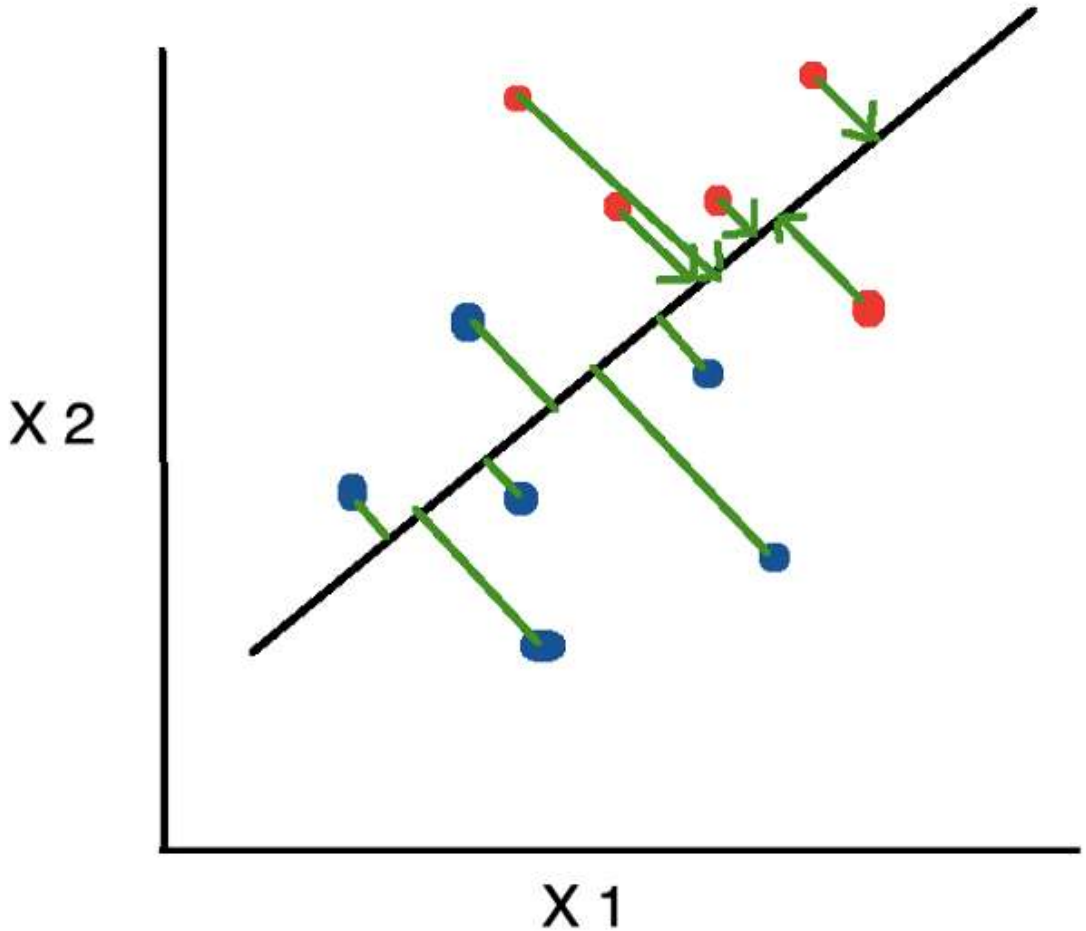
If we'd like to reduce the number of dimensions down to 1, one approach would be to project everything on to the x-axis.

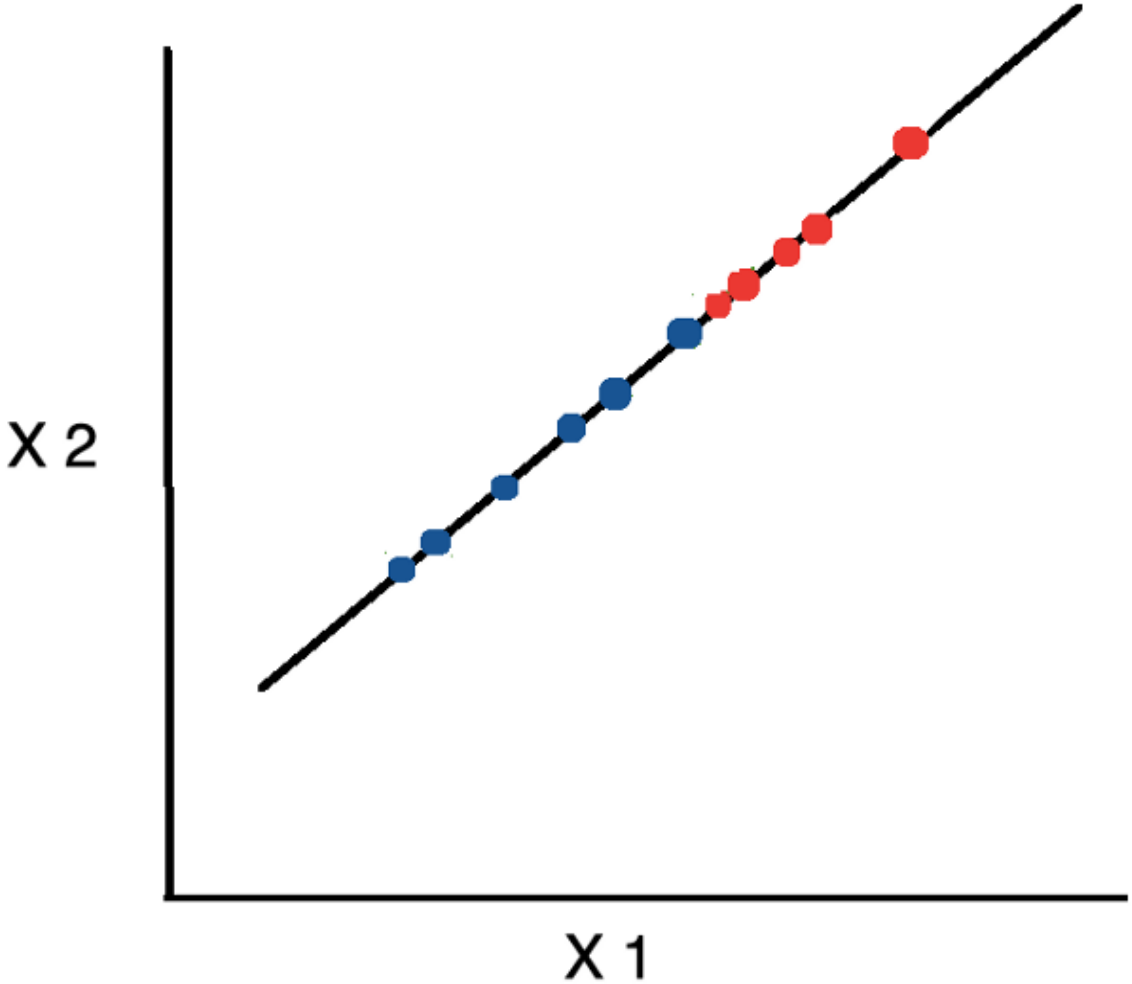


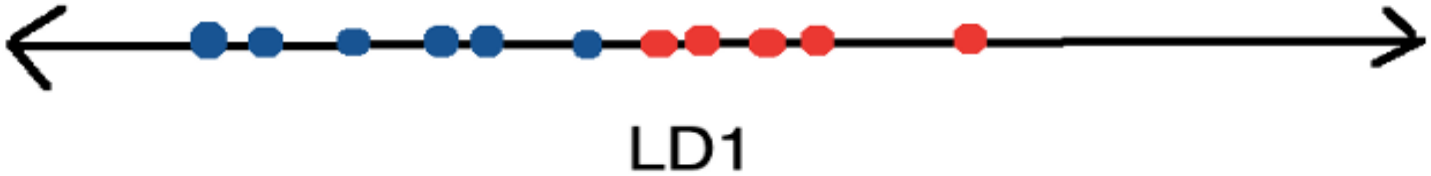
This is bad because it ignores any useful information provided by the second feature.



Linear Discriminant Analysis, or LDA, uses the information from both features to create a new axis and projects the data on to the new axis in such a way as to minimizes the variance and maximizes the distance between the means of the two classes.




X 2
X 1


LD1

- Unlike PCA that calculates eigenvalues of the covariance matrix of the data set, **LDA calculates eigenvalues and eigenvectors within a class and inter-class scatter matrices.**
- Below are the steps to be followed:

1. Calculate the mean vectors for the individual classes.
2. Calculate intra-class and inter-class scatter matrices.
3. Calculate eigenvalues and eigenvectors for S_W^{-1} and S_B, where S_W is the intra-class scatter matrix and S_B is the inter-class scatter matrix

$$S_W = \sum_{i=1}^{c} S_i;$$

$$S_i = \sum_{x \in D_i}^{n} (x - m_i)(x - m_i)^T$$

where, m_i is the mean vector of the i-th class

$$S_B = \sum_{i=1}^{c} N_i (m_i - m)(m_i - m)^T$$

where, mi is the sample mean for each class, m is the overall mean of the data set, *Ni* is the sample size of each class

4. Identify the top 'k' eigenvectors having top 'k' eigenvalues

2. Feature Subset Selection

Feature Subset Selection

- Feature selection is arguably the most critical pre-processing activity in any machine learning project.

- It **intends to select a subset of system attributes or features which makes a most meaningful contribution in a machine learning activity.**

- **Ex.**, we are trying to predict the weight of students based on past information about similar students, which is captured in a **'student weight'** data set.

Example

Roll Number	Age	Height	Weight
12	12	1.1	23
14	11	1.05	21.6
19	13	1.2	24.7
32	11	1.07	21.3
38	14	1.24	25.2
45	12	1.12	23.4

→

Age	Height	Weight
12	1.1	23
11	1.05	21.6
13	1.2	24.7
11	1.07	21.3
14	1.24	25.2
12	1.12	23.4

FIG. 4.8
Feature selection

- The student weight data set has features such as **Roll Numbcr, Age. Height, and Weight.**

- We can well understand that **roll number can have no behavior, whatsoever, in predicting student weight.**

- So we **can eliminate the feature roll number and build a feature subset to be considered in this machine learning problem**.

- **The subset of features is expected to give better results than the full set.**

Objectives of feature selection

- Having faster and cost-effective (i.e. less need for computational resourses) learning model.
- Improving the efficiency of the learning model.
- Having better understanding of the underlying model that generated the data.

Key driver for feature selection

- **Feature relevance**
- **Feature redundancy**

Feature relevance

- **In supervised learning**, the input data set which is the training data set, has a class label attached.
- A model is trained based on the training data set — so that the inducted model can assign class labels to new, unlabelled data.
- **Each of the predictor variables, is expected to contribute information to decide the value of the class label.**
- In case **a variable is not contributing any information, it is said to be irrelevant.**
- In case the information contribution for **prediction is very little, the variable is said to be weakly relevant.**
- Remaining variables, which make a significant contribution to the prediction task are said to be strongly relevant variables.

- **In unsupervised learning**, there is **no training data set or labeled data.**

- Grouping of similar data instances are done and **similarity of data instances are evaluated based on the value of different variables.**

- Certain variables do not contribute any useful information for deciding the similarity of difference of data instances.

- Hence, those variables make no significant information contribution in the grouping process.

- These **variables are marked as irrelevant variables in the context of the unsupervised machine learning task**.

- example of the student data set that we discussed at the beginning of this section.

- Roll number of a student doesn‘t contribute any significant information in predicting what the Weight of a student would be.

- Similarly, if we are trying to group together students with similar academic capabilities, Roll number can really not contribute any information whatsoever.

- So, in **context of the supervised task of predicting student Weight** or **the unsupervised task of grouping students with similar academic merit**, **the variable Roll number is quite irrelevant.**

Feature Redundancy (unnecessary)

- A feature may contribute information which is similar to the information contributed by one or more other features.
- **For example**. in the weight prediction problem referred earlier in the section, both the features Age and Height contribute similar information.
- This is because with an increase in Age, Weight is expected to increase. Similarly, with the increase of Height also Weight is expected to increase. Also, Age and Height increase with each other.
- **So, in context of the Weight prediction problem, Age and Height contribute similar information.**
- In other words, irrespective of whether the feature height is present as a part of the feature subset, the learning model will give almost same results.

- In the same way, without age being part of the predictor variables, the outcome of the learning model will be more or less same.

- In this kind of a **situation when one feature is similar to another feature, the feature is said to be potentially redundant in the context of the learning problem.**

- The **main objective of feature selection is to remove all features which are irrelevant and take a representative subset of the features which are potentially redundant.**

- This leads to a meaningful feature subset in context of a specific learning task.

Extra Topic

Bayesian Concept Learning

- **Importance of Bayesian methods,**
- **Bayes' theorem,**
- **Concept learning through Bayes' theorem,**
- **Bayesian Belief Network**

Probability

- **Probability is the measure of the likelihood that an** event will occur.
- Probability is quantified as a number between 0 and 1 (where 0 indicates impossibility and 1 indicates certainty).
- **Probability= Likelihood= Chance**
- **<u>Three Term</u>**

(1)Experiment

- A process that leads to the occurrence of one(and only one) of several possible observation. (eg- tossing a coin)

(2)Outcome

- A particular result of the experiment. (eg – tossing a coin => head & tail)

Experiment	Outcome

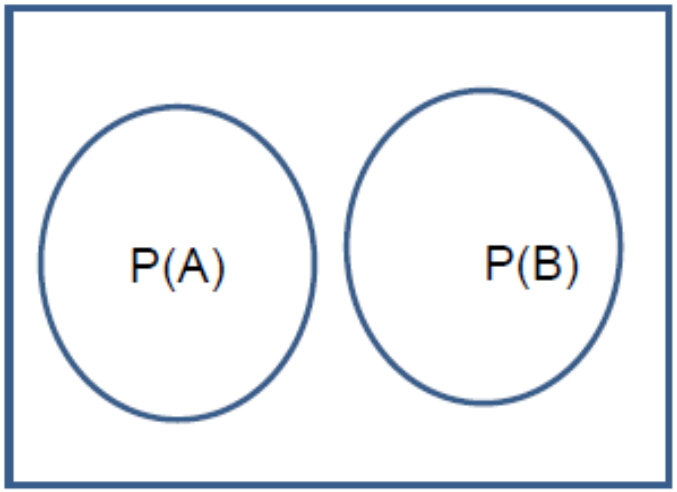
(3)Event

- A **collection of one of more outcomes of an experiment**.
  - 1st time=> 200 people tossing a coin
  - 2nd time => 500 people tossing a coin

- **Example**
- A simple example is the toss of a fair (unbiased) coin.
- Since the two outcomes are equally probable, the probability of "heads" equals the probability of "tails“, so the probability is 1/2 (or 50%) chance of either "heads" or "tails".

Probability Concept

Independent Case



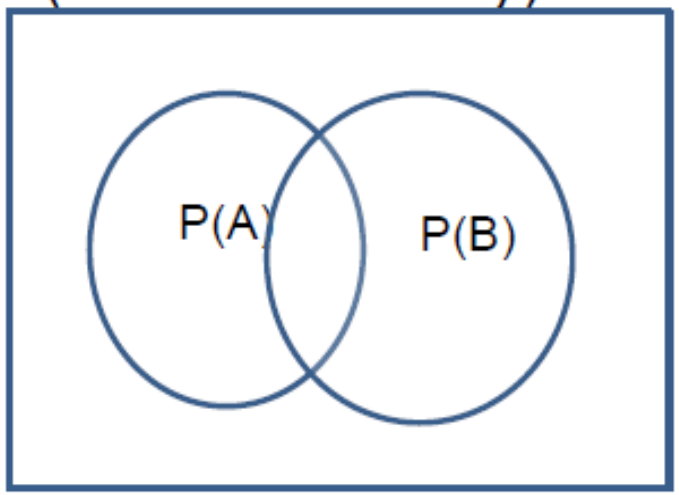
Addition

P(A or B)= P(A)+P(B)

Multiplication

*P(A and B)= P(A) * P(B)*

Dependent Case
(Joint Probability)



Addition

P(A or B)= P(A)+P(B)-P(A and B)

Multiplication

*P(A and B)= P(A) * P(B|A)*

Joint Probability & Conditional Probability

- **Joint Probability** is the likelihood that two or more events will happen **at the same time.**

- A **conditional probability** is the likelihood that an event will happen given that another event has already happened.
- **A conditional probability measures the probability of an** event given that (by assumption, presumption, assertion or evidence) another event has occurred.
- If the event of interest is ***A and the event B is known or assumed to have* occurred, "the conditional probability of *A given B*", *or* "the probability of *A under the condition B*", *is usually* written as $P(A|B)$**

History of Bayes' Theorem

- In the 18th Century , **Reverend Thomas Bayes, an English Presbyterian minister, wonder about this question:**

- **"Does God really exist?"**

- Being interested in the mathematics, **he attempt to develop a formula to arrive at the probability that God does exist based on the evidence that was available to him on earth.**

- Later, **Laplace refined Bayes' work and gave it the name "Bayes' Theorem".**

Bayes' Theorem

- Bayes' Theorem (also known as Bayes' rule) is a deceptively **simple formula used to calculate conditional probability.**

- In probability theory and statistics, Bayes' theorem (alternatively Bayes' law or Bayes'rule) **describes the probability of an event, based on conditions that might be related to the event**.

- Bayes' Theorem is a **method of revising a probability**, given that additional information is obtained. For two event:

$$P(A/B) = \frac{P(B/A)P(A)}{P(B)}$$

- Show the **Relation between one conditional probability and its inverse.**

- Provide a **mathematical rule for revising an estimate or forecast in light of experience and observation.**

$$P(A/B) = \frac{P(B/A)P(A)}{P(B)}$$

- Where **A and B are events**:
- **P(A) and P(B) are the probabilities** of A and B without regard to each other.
- **P(A | B), a conditional probability**, is the probability of observing event A given that B is true.
- **P(B | A) is the probability of observing event B given that A is true**.

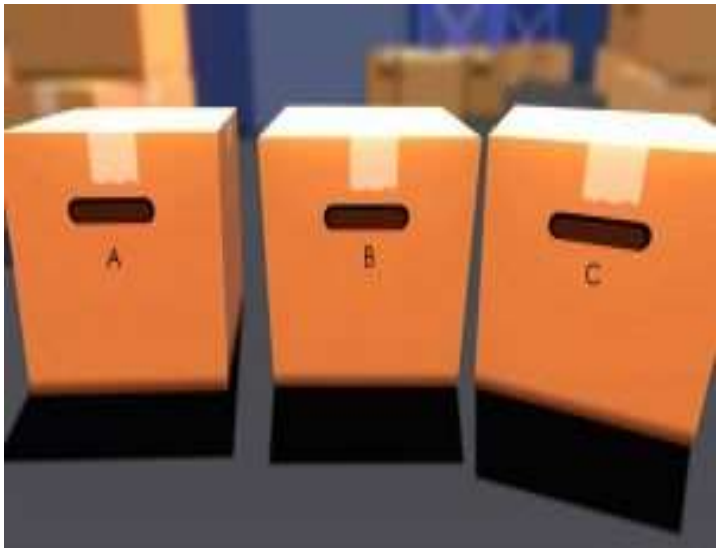
Concept of Prior, Posterior and likelihood

$$\underbrace{P(A|B)}_{\text{Posterior}} = \frac{\overbrace{P(B|A)}^{\text{Likelihood}} * \overbrace{P(A)}^{\text{Prior}}}{\underbrace{P(B)}_{\text{Evidence}}}$$

- **posterior = likelihood * prior / evidence**
- **Posterior Probability (A|B):** The probabilities after testing are called Posterior Probability.
- **likelihood P(B|A) :** This is the probability of seeing the data given a *certain* hypothesis.
- **Prior Probability P(A)**: The initial probability before any testing called the prior probability.
- **Evidence P(B) :** This is simply the probability of the data under *any* hypothesis. It is called the evidence or normalizing constant.

Bayes' Theorem Example #1

- There are 3 boxes labeled A, B, and C:
  - **Box A** contains 2 red and 3 black balls
  - **Box B** contains 3 red and 1 black ball
  - **And box C** contains 1 red ball and 4 black balls
- The three boxes are identical and have an equal probability of getting picked.
- Consider that a red ball is chosen.
- Then **what is the probability that this red ball was picked out of box A?**



Solution

- Let **E denote the event that a red ball is chosen** and A, B, and C denote that the respective box is picked.
- We are required to calculate **the conditional probability P(A | E).**

```
We have prior probabilities P(A) = P(B) = P (C) = 1 / 3, since all boxes have equal
probability of getting picked.


P(E|A) = Number of red balls in box A / Total number of balls in box A = 2 / 5
Similarly, P(E|B) = 3 / 4 and P(E|C) = 1 / 5


Then evidence P(E) = P(E|A)*P(A) + P(E|B)*P(B) + P(E|C)*P(C)
                   = (2/5) * (1/3) + (3/4) * (1/3) + (1/5) * (1/3) = 0.45
Therefore, P(A|E) = P(E|A) * P(A) / P(E) = (2/5) * (1/3) / 0.45 = 0.296
```

Bayes' Theorem Example #2

- You might be interested in finding out a patient's probability of having liver disease if they are an alcoholic. "Being an alcoholic" is the **test** (kind of like a litmus test) for liver disease.
- **A** could mean the event "Patient has liver disease." Past data tells you that 10% of patients entering your clinic have liver disease.
  - **P(A) = 0.10.**
- **B** could mean the litmus test that "Patient is an alcoholic." Five percent of the clinic's patients are alcoholics.
  - **P(B) = 0.05.**
- You might also know that among those patients diagnosed with liver disease, 7% are alcoholics.
  - This is your **B | A:** the probability that a patient is alcoholic, given that they have liver disease, is 7%.

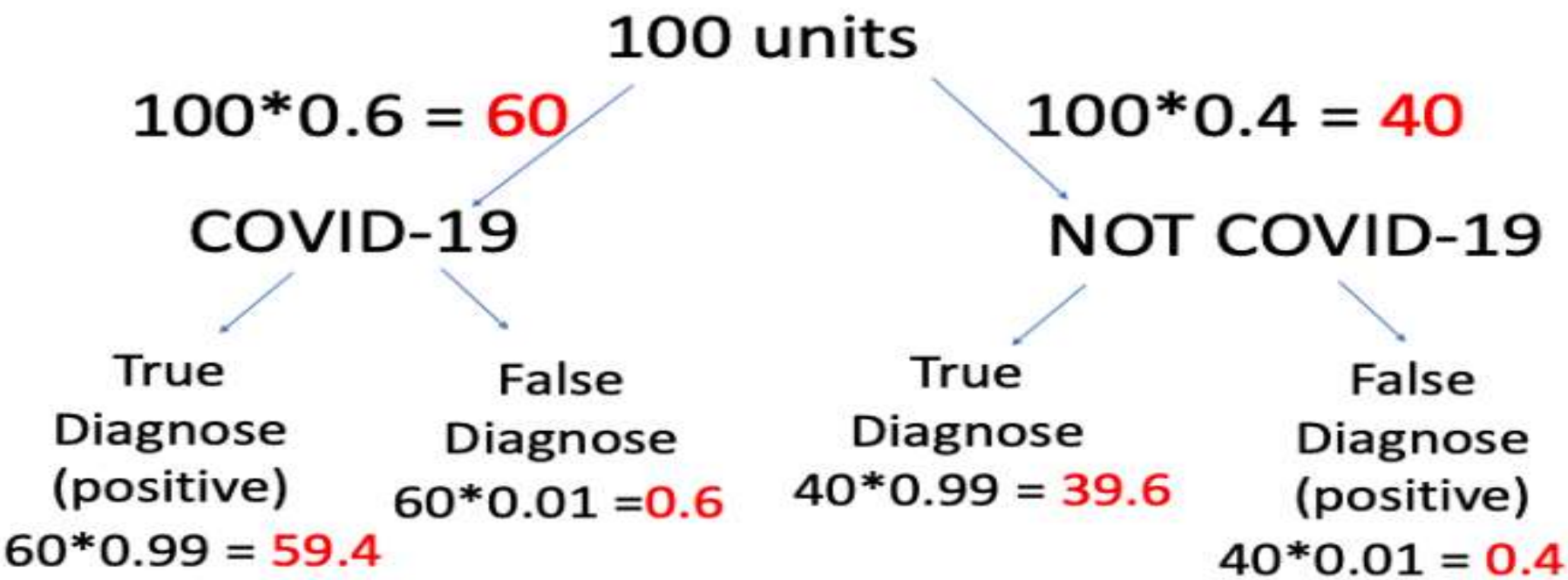
- Bayes' theorem tells you:

$$\mathbf{P(A|B) = (0.07 * 0.1)/0.05 = 0.14}$$

- In other words, if the patient is an alcoholic, their chances of having liver disease **is 0.14 (14%).**

Bayes' Theorem Example #3

- Covid-19 tests are common nowadays, but some results of tests are not true. Let's assume; a diagnostic test has 99% accuracy and 60% of all people have Covid-19. If a patient tests positive, what is the probability that they actually have the disease?



- **The total units which have positive results**= 59.4 + 0.4 = 59.8
- 59.4 units (true positive) is 59.8 units means 99.3% = 0.993 probability

- **With Bayes';**

$$P(covid19|positive) = \frac{P(positive|covid19) * P(covid19)}{P(positive)}$$

- **P(positive | covid19)** = 0.99
- **P(covid19)** = 0.6
- **P(positive)** = 0.6*0.99+0.4*0.01=0.598

$$P(covid19|positive) = \frac{0.99 * 0.6}{0.598} = 0.993$$

Naive Bayes classifier

- Naive Bayes is a family of algorithms based on applying Bayes' theorem with a **strong (naive) assumption**, that **every feature is independent of the others, in order to predict the category of a given sample.**

- They are **probabilistic classifiers**, therefore **will calculate the probability of each category using Bayes' theorem,** and the **category with the highest probability will be the output**.

- Naive Bayes classifiers have been successfully applied to many domains, particularly **Natural Language Processing (NLP).**

- They are also known for creating simple yet well-performing models, especially in the fields of **document classification and disease prediction**.

- **For example**, a fruit may be considered to be an apple if it is red, round, and about 3 inches in diameter.

- Even if these features depend on each other or upon the existence of the other features, all of these properties independently contribute to the probability that this fruit is an apple and **that is why it is known as 'Naive'.**

Bayesian Classification: Why?

- **A statistical classifier:** performs *probabilistic prediction, i.e.,* predicts class membership probabilities

- **Foundation:** Based on Bayes' Theorem.

- **Performance:** A simple Bayesian classifier, *naïve Bayesian classifier*, has comparable performance with decision tree and selected neural network classifiers

- **Incremental**: Each training example can incrementally increase/decrease the probability that a hypothesis is correct — prior knowledge can be combined with observed data

Choosing Hypotheses

- *Maximum Likelihood* hypothesis:

$$h_{ML} = \arg\max_{h \in H} P(d \mid h)$$

- Generally we want the most probable hypothesis given training data. This is the *maximum a posteriori* hypothesis:
  - Useful observation: it does not depend on the denominator P(d)

$$h_{MAP} = \arg\max_{h \in H} P(h \mid d)$$

Bayesian Classifiers

- Consider each attribute and class label as random variables

- Given a record with attributes $(A_1, A_2, \ldots, A_n)$
  - Goal is to predict class C
  - Specifically, we want to find the value of C that maximizes $P(C \mid A_1, A_2, \ldots, A_n)$

- Can we estimate $P(C \mid A_1, A_2, \ldots, A_n)$ directly from data?

Bayesian Classifiers

- **Approach:**
  - compute the posterior probability $P(C \mid A_1, A_2, \ldots, A_n)$ for all values of C using the Bayes theorem

$$\boxed{P(C \mid A_1 A_2 \ldots A_n) = \frac{P(A_1 A_2 \ldots A_n \mid C) P(C)}{P(A_1 A_2 \ldots A_n)}}$$

  - Choose value of C that maximizes

    $P(C \mid A_1, A_2, \ldots, A_n)$

  - Equivalent to choosing value of C that maximizes

    $P(A_1, A_2, \ldots, A_n \mid C)\, P(C)$

- How to estimate $P(A_1, A_2, \ldots, A_n \mid C)$?

Naïve Bayes Classifier

- Assume independence among attributes A_i when class is given:
  - $P(A_1, A_2, \ldots, A_n \mid C) = P(A_1 \mid C_j)\, P(A_2 \mid C_j) \ldots P(A_n \mid C_j)$
  - Can estimate $P(A_i \mid C_j)$ for all A_i and C_j.
  - This is a simplifying assumption which may be violated in reality

- **The Bayesian classifier that uses the Naïve Bayes assumption and computes the MAP hypothesis is called Naïve Bayes classifier**

$$\boxed{c_{Naive\ Bayes} = \arg\max_c P(c)P(\mathbf{x} \mid c) = \arg\max_c P(c)\prod_i P(a_i \mid c)}$$

Example.

- Let us assume that we want to predict the outcome of a football world cup match on the basis of the past performance data of the playing teams. We have training data available (refer Fig. 6.3) for actual match outcome.
- While four parameters are considered - Weather Condition (Rainy, Overcast, or Sunny).
- how many matches won were by this team out of the last three matches (one match, two matches, or three matches), Humidity Condition (High or Normal), and whether they won the toss (True or False).
- Using Naive Bayesian, you need to classify the conditions when this team wins and then **predict the probability of this team winning a particular match when Weather Conditions = Rainy. They won two of the last three matches, Humidity = Normal and they won the toss in the particular match.**

Weather Condition	Wins in last 3 matches	Humidity	Win toss	Won match?
Rainy	3 wins	High	FALSE	No
Rainy	3 wins	High	TRUE	No
OverCast	3 wins	High	FALSE	Yes
Sunny	2 wins	High	FALSE	Yes
Sunny	1 win	Normal	FALSE	Yes
Sunny	1 win	Normal	TRUE	No
OverCast	1 win	Normal	TRUE	Yes
Rainy	2 wins	High	FALSE	No
Rainy	1 win	Normal	FALSE	Yes
Sunny	2 wins	Normal	FALSE	Yes
Rainy	2 wins	Normal	TRUE	Yes
OverCast	2 wins	High	TRUE	Yes
OverCast	3 wins	Normal	FALSE	Yes
Sunny	2 wins	High	TRUE	No

FIG. 6.3
Training data for the Naïve Bayesian method

Naïve bayes classifer steps

Step 1: First construct a frequency table. A frequency table is drawn for each attribute against the target outcome. For example, in Figure 6.3, the various attributes are (1) Weather Condition, (2) How many matches won by this team in last three matches, (3) Humidity Condition, and (4) whether they won the toss and the target outcome is will they win the match or not?

Step 2: Identify the cumulative probability for 'Won match = Yes' and the probability for 'Won match = No' on the basis of all the attributes. Otherwise, simply multiply probabilities of all favourable conditions to derive 'YES' condition. Multiply probabilities of all non-favourable conditions to derive 'No' condition.

Step 3: Calculate probability through normalization by applying the below formula

$$P(\text{Yes}) = \frac{P(\text{Yes})}{P(\text{Yes}) + P(\text{No})}$$

$$P(\text{No}) = \frac{P(\text{No})}{P(\text{Yes}) + P(\text{No})}$$

P(Yes) will give the overall probability of favourable condition in the given scenario.

P(No) will give the overall probability of non-favourable condition in the given scenario.

Step-1 Construct Frequency Table

	Won Match	
Weather condition	Yes	No
Sunny	3	2
OverCast	4	0
Rainy	2	3
Total	9	5

	Won Match	
Humidity	Yes	No
High	3	4
Normal	6	1
Total	9	5

	Won Match	
Wins in last 3 matches	Yes	No
3 wins	2	2
1 win	4	2
2 wins	3	1
Total	9	5

	Won Match	
Win toss	Yes	No
FALSE	6	2
TRUE	3	3
Total	9	5

FIG. 6.4
Construct frequency table

Step-2 Identify Cumulative probability for won match= yes and won match= no

$$P(\text{Win match}|a_1 \cap a_2 \cap a_3 \cap a_4)$$

$$= \frac{P(a_1|\text{Win match})P(a_2|\text{Win match})P(a_3|\text{Win match})P(a_4|\text{Win match})P(\text{Win match})}{P(a_1)P(a_2)P(a_3)P(a_4)}$$

$$= 2/9*4/9*6/9*9/14$$

$$= 0.014109347$$

This should be compared with

$$P(!\text{Win match}|a_1 \cap a_2 \cap a_3 \cap a_4)$$

$$= \frac{P(a_1|!\text{Win match})P(a_2|!\text{Win match})P(a_3|!\text{Win match})P(a_4|!\text{Win match})P(!\text{Win match})}{P(a_1)P(a_2)P(a_3)P(a_4)}$$

$$= 3/5*2/5*1/5*5/14$$

$$= 0.010285714$$

Step-3 Normalizing the Probability

$$P(\textbf{Win match}) = \frac{P(\textbf{Win match})}{P(\textbf{Win match}) + P(\textbf{!Win match})}$$
$$= \frac{0.014109347}{0.014109347 + 0.010285714}$$
$$= 0.578368999$$

$$P(\textbf{!Win match}) = \frac{P(\textbf{!Win match})}{P(\textbf{Win match}) + P(\textbf{!Win match})}$$
$$= \frac{0.010285714}{0.014109347 + 0.010285714}$$
$$= 0.421631001$$

Conclusion: This shows that there is 58% probability that the team will win if the above conditions become true for that particular day. Thus, Naïve Bayes classifier provides a simple yet powerful way to consider the influence of multiple attributes on the target outcome and refine the uncertainty of the event on the basis of the prior knowledge because it is able to simplify the calculation through independence assumption.

Strengths and Weaknesses of Bayes Classifiers

Strengths	Weakness
Simple and fast in calculation but yet effective in result	The basis assumption of equal importance and independence often does not hold true
In situations where there are noisy and missing data, it performs well	If the target dataset contains large numbers of numeric features, then the reliability of the outcome becomes limited
Works equally well when smaller number of data is present for training as well as very large number of training data is available	Though the predicted classes have a high reliability, estimated probabilities have relatively lower reliability
Easy and straightforward way to obtain the estimated probability of a prediction	

- Example: Play Tennis
  - Given a new instance, predict its label

    $\mathbf{x}'$=(Outlook=*Sunny*, Temperature=*Cool*, Humidity=*High*, Wind=*Strong*)

PlayTennis: training examples

Day	Outlook	Temperature	Humidity	Wind	PlayTennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Applications of Navie Bayes classifier

- **Text classification:** Naive Bayes classifier is among the most successful known algorithms **for learning to classify text documents**.

- It classifies the document where the probability of classifying the text is more.

- It uses the above algorithm **to check the variation and combination of the probability of classifying a document under a particular "Title".**

- It has various applications in document categorization, language detection, and sentiment detection, which are very useful for traditional retailers, e-retailors, and other businesses on judging the sentiments of their clients on the basis of keywords in feedback forms, social media comments, etc.

- **Spam filtering:** Spam filtering is the best known **use of Naive Bayesian text classification.**

- Presently, almost all the email providers have this as a built-in functionality, which makes use of a Naive Bayes **classifier to identify spam email on the basis of certain conditions and also the probability of classifying an email as "Spam".**

- Naïve Bayesian spam sifting has turned into a mainstream mechanism to **recognize illegal a spam email from** an honest-to-goodness email (sometimes called 'ham'),Users can also install separate email filtering programmes.

- Server-side email filters such as **DSPAM,Spam Assassin, Spam Bayes, and ASSP make use of Bayesian spam filtering techniques**, and the functionality is sometimes embedded within the mail server software itself.

- **Hybrid Recommender System:** It uses Naive Bayes classifier and collaborative filtering.

- Recommender systems (used by e-retailors like eBay, Alibaba, Target, Flipkart, etc.) apply **machine learning and data mining techniques for filtering unseen information and can predict whether a user would like a given resource.**

- **For example**, when we log in to these retailer websites, on the basis of the usage of texts used by the login and the historical data of purchase, it automatically recommends the product for the particular login persona.

- One of the algorithms is combining a Naive Bayes classification approach with collaborative filtering, and experimental results show that **this algorithm provides better performance regarding accuracy and coverage than other algorithms.**

- **Online Sentiment Analysis:** The online applications use supervised machine learning (Naive Bayes) and useful computing.

- In the case of sentiment analysis, let us assume there are **three sentiments such as nice, nasty, or neutral,** and Naive Bayes classifier is used to distinguish between them. **Simple emotion modeling combines a statistically based classifier with a dynamical model.**

- The Naive Bayes classifier employs **'single words**' and '**word pairs**' like features and determines the sentiments of the users.

- It allocates user utterances into nice, nasty, and neutral classes**, labeled as +1,-1, and 0, respectively.**

- This binary output drives a simple first-order dynamical system, whose emotional state represents the simulated emotional state of the experiment's representation.

Bayesian Belief Network

- Bayesian belief network is key computer technology for **dealing with probabilistic events and to solve a problem which has uncertainty**.

- We can define a Bayesian network as:
- "**A Bayesian network is a probabilistic graphical model which represents a set of variables and their conditional dependencies using a directed acyclic graph**."

- It is also called a **Bayes network, belief network, decision network**, or **Bayesian model**.

- Bayesian networks are probabilistic, because these networks are built from a **probability distribution**, and also use probability theory for prediction and anomaly detection.

- Real world applications are probabilistic in nature, and to represent the relationship between multiple events, we need a Bayesian network.

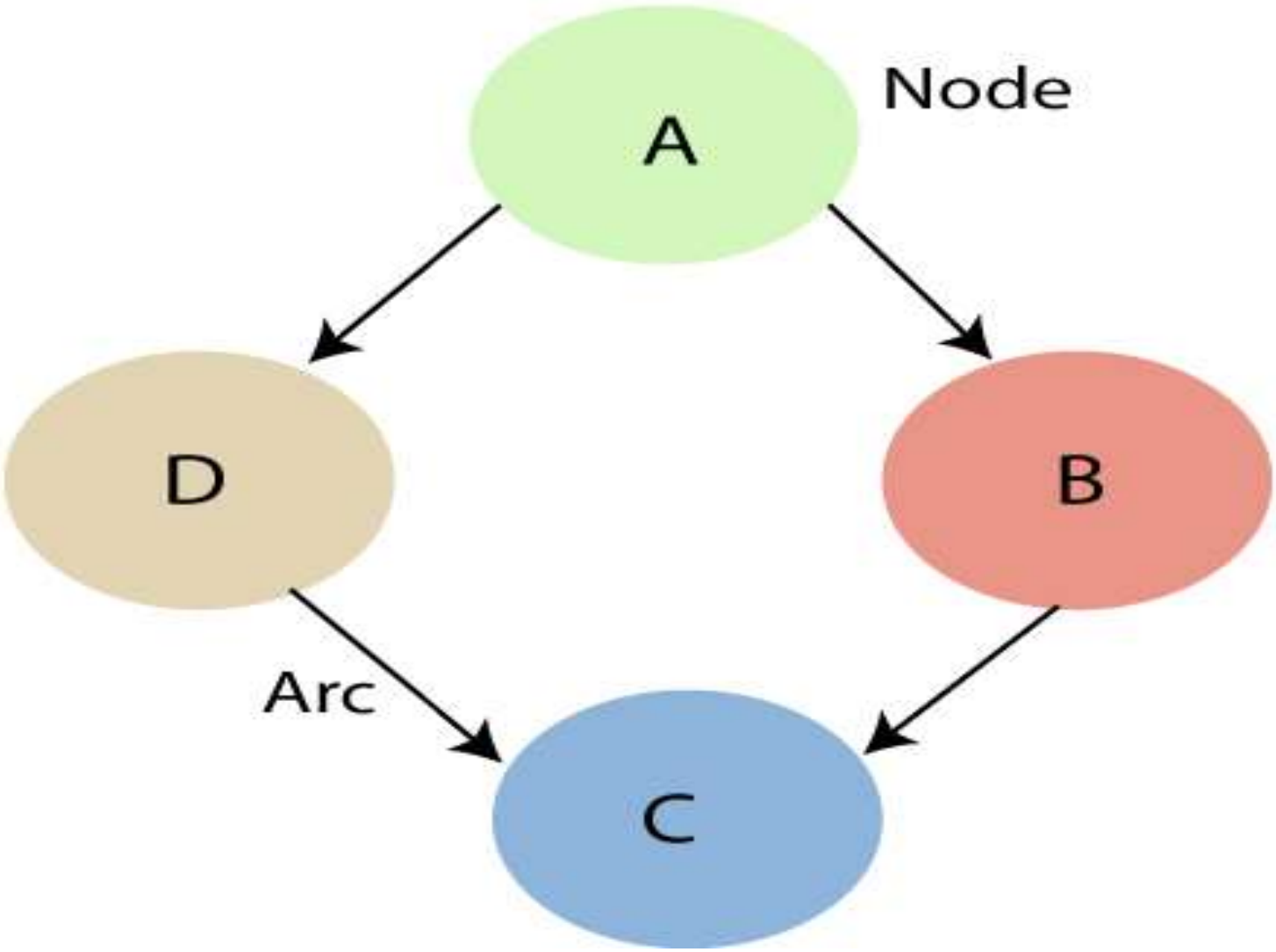
- It can also be used in various tasks including **prediction, anomaly detection, diagnostics, automated insight, reasoning, time series prediction**, and **decision making under uncertainty**.

- Bayesian Network can be used for building models from data and experts opinions, and it consists of two parts:

  - **Directed Acyclic Graph.**
  - **Table of conditional probabilities.**

- The generalized form of Bayesian network that represents and solve decision problems under uncertain knowledge is known as an **Influence diagram**

- **A Bayesian network graph is made up of nodes and Arcs (directed links), where:**



- Each **node** corresponds to the random variables, and a variable can be **continuous** or **separate**.

- **Arc or directed arrows** represent the causal relationship or conditional probabilities between random variables.

- These directed links or arrows connect the pair of nodes in the graph.

- These links represent that one node directly influence the other node, and **if there is no directed link that means that nodes are independent with each other**
  - **In the diagram, A, B, C, and D are random variables represented by the nodes of the network graph.**
  - **If we are considering node B, which is connected with node A by a directed arrow, then node A is called the parent of Node B.**
  - **Node C is independent of node A.**

- The Bayesian network has mainly two components:
  - **Causal Component**
  - **Actual numbers**

- Each node in the Bayesian network has condition probability distribution $\mathbf{P(X_i \mid Parent(X_i))}$, which determines the effect of the parent on that node.

- Bayesian network is based on **Joint probability distribution** and **conditional probability.**